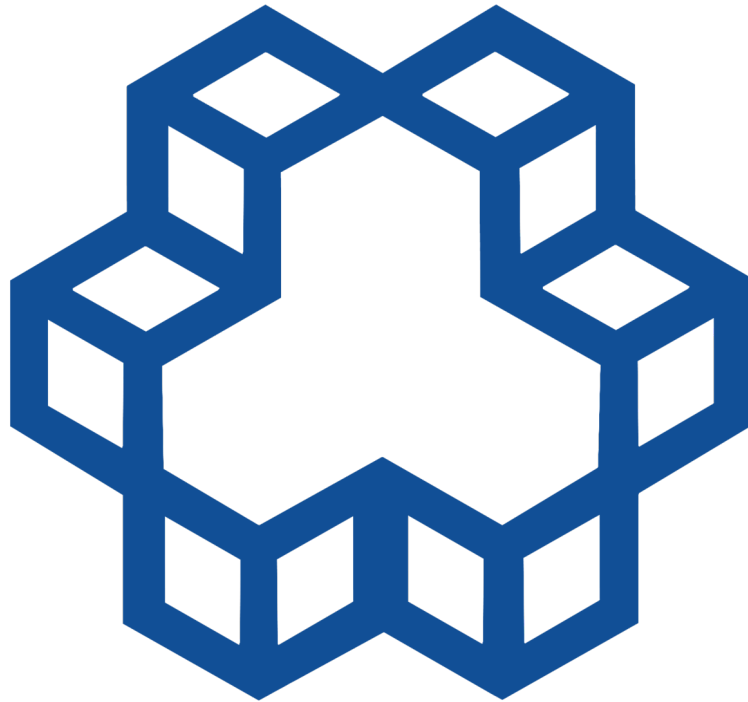


گزارش کار کامل پروژه پایگاه داده

سامانه جامع رزرو، خرید و مدیریت بلیط مسابقات ورزشی



1. مقدمه و معرفی مسئله
2. خلاصه معماری کل سیستم
3. نقشه نیازمندی‌ها و امکانات پیاده‌سازی شده
4. فاز اول: طراحی پایگاه داده و ERD
5. فاز دوم: پیاده‌سازی SQL، داده اولیه، کوئری‌ها، توابع و ایندکس‌ها
6. فاز سوم: Backend API، احراز هویت، Redis و منطق تراکنشی
7. فاز چهارم: رابط کاربری، جستجوی پیشرفته، Elasticsearch و Autocomplete
8. تست نهایی پروژه و سناریوهای اجرا
9. راهنمای نصب و اجرای پروژه
10. جمع‌بندی و کارهای قابل توسعه

۱. مقدمه و معرفی مسئله

هدف این پروژه طراحی و پیاده‌سازی یک سامانه جامع برای رزرو و خرید بلیط مسابقات ورزشی است. کاربران عادی یا تماشاگران می‌توانند مسابقات را جستجو کنند، بلیط مناسب را بر اساس شهر، رشته ورزشی، جایگاه، قیمت، تاریخ و ظرفیت باقی‌مانده انتخاب کنند، آن را به صورت موقت رزرو کنند، پرداخت انجام دهند و در صورت نیاز درخواست کنسلی یا استرداد وجه ثبت کنند. از طرف دیگر، کاربران پشتیبان می‌توانند گزارش‌های ثبت‌شده توسط کاربران و وضعیت رزروها را مدیریت کنند.

در نسخه نهایی پروژه، سیستم فقط یک پایگاه داده ساده نیست؛ بلکه یک جریان کامل از طراحی داده تا Backend، امنیت، کش، OTP، پرداخت، Elasticsearch، Refund و UI پیاده‌سازی شده است. بنابراین گزارش حاضر هر چهار فاز را به شکل یکپارچه توضیح می‌دهد تا مسیر توسعه پروژه از ERD تا رابط کاربری قابل مشاهده باشد.

۱-۱. بازیگران اصلی سیستم

- تماشاگر یا Spectator: کاربری که ثبت‌نام می‌کند، وارد سیستم می‌شود، بلیط را جستجو و رزرو می‌کند، پرداخت انجام می‌دهد، تاریخچه خرید را می‌بیند و گزارش مشکل ثبت می‌کند.
- پشتیبان یا Support: کاربری با نقش پشتیبان که به گزارش‌های کاربران رسیدگی می‌کند و وضعیت رزروها را از پنل پشتیبانی مدیریت می‌کند.
- سیستم Backend: مسئول کنترل امنیت، اعتبارسنجی، اجرای Queryها، مدیریت تراکنش‌های دیتابیس، اتصال به Redis و Elasticsearch و پاسخ JSON است.
- سیستم پایگاه داده: PostgreSQL که جداول نرمال‌شده، کلیدهای خارجی، محدودیت‌ها، داده اولیه، کوئری‌های تحلیلی، توابع و ایندکس‌ها را نگهداری می‌کند.

۲. خلاصه معماری کل سیستم

معماری پروژه به صورت چندلایه طراحی شده است. لایه Frontend فقط وظیفه نمایش و ارسال درخواست را دارد. Backend درخواست‌ها را دریافت می‌کند، token و role کاربر را بررسی می‌کند، Queryهای SQL را بدون ORM اجرا می‌کند، در صورت نیاز از Redis برای OTP و cache استفاده می‌کند و برای جستجوی متنی از Elasticsearch کمک می‌گیرد. داده اصلی همچنان در PostgreSQL نگهداری می‌شود.

وظیفه	فناوری	لایه
نمایش داشبورد، فرم‌ها، کارت‌های نتیجه، مدیریت token در localStorage و ارسال درخواست به API	HTML / CSS / JavaScript	Frontend
تعریف routeها، controllerها، middlewareها، اعتبارسنجی و تولید پاسخ JSON	Node.js / Express	Backend
پسورد ابتدا چک می‌شود، سپس OTP ارسال و تایید می‌شود، بعد JWT ساخته می‌شود	JWT + bcrypt + Redis OTP	Authentication
نگهداری داده اصلی، روابط، تراکنش‌ها، محدودیت‌ها، queryها و functionها	PostgreSQL	Database
ذخیره موقت OTP و cache کردن داده‌های پرتکرار مثل شهرها و جستجوی بلیط	Redis	Cache / OTP

جستجوی متنی روی تیم، شهر، ورزشگاه، ورزش، امکانات و autocomplete	Elasticsearch + Docker	Search Engine
---	------------------------	---------------

Frontend UI → Express Routes → Controllers → PostgreSQL

↓
Redis

↓
Elasticsearch

۳. نقشه نیازمندی‌ها و امکانات پیاده‌سازی شده

جدول زیر نشان می‌دهد هر نیاز اصلی پروژه در کدام بخش پیاده‌سازی شده است. این جدول برای دفاع از پروژه مفید است، چون مشخص می‌کند هر قابلیت فقط در ظاهر UI نیست و در Backend و Database هم پشتیبانی شده است.

نیازمندی	وضعیت	محل پیاده‌سازی
طراحی ERD و موجودیت‌ها	انجام شده	فاز ۱، گزارش طراحی و ساختار جداول
نرمال‌سازی تا 3NF	انجام شده	جدا شدن City، Venue، Sport، Match، Ticket، Payment و سایر موجودیت‌ها
ساخت جداول PostgreSQL	انجام شده	فاز ۲، schema دیتابیس sports_ticketing_db
داده اولیه حداقل ۱۰ رکورد برای جداول	انجام شده	اسکرپت‌های INSERT
۲۲ کوئری تحلیلی	انجام شده	SQL queries برای فروش، رزرو، کاربران، ظرفیت، گزارش و refund
۸ تابع یا procedure	انجام شده	توابع تحلیلی و جستجویی دیتابیس
Indexing و Query Optimization	انجام شده	ایندکس روی فیلدهای جستجو و کلیدهای خارجی
Backend بدون ORM	انجام شده	Node.js + Express + pg با SQL خام و parameterized
JWT Authentication	انجام شده	token و middleware authenticate در login
Password + OTP Login	انجام شده	اول bcrypt password، سپس OTP در Redis، سپس JWT
Reservation temporary	انجام شده	کاهش ظرفیت در transaction و expiry_time

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

Auto-expire unpaid reservations	انجام شده	background job و برگشت ظرفیت
Payment	انجام شده	status paid و پرداخت endpoint
Refund	انجام شده	endpoint refund، محاسبه جریمه، status refunded/cancelled
Change ticket / seat	انجام شده	تغییر ticket_id رزرو پرداخت نشده و اصلاح capacity
Support report management	انجام شده	پنل پشتیبان برای گزارش ها
Support reservation management	انجام شده	مشاهده و تغییر status رزروها توسط support
Advanced ticket filters	انجام شده	city، sport، seat، min/max price، date، min capacity
Autocomplete و Elasticsearch	انجام شده	index-tickets و ...=search/tickets?q
Frontend Dashboard	انجام شده	صفحات Home، Auth، Profile، Tickets، Booking، History، Support، ElasticSearch

۴. فاز اول: طراحی پایگاه داده و ERD

فاز اول پایه کل پروژه بود. در این فاز ابتدا نیازمندی‌های سیستم رزرو بلیط تحلیل شد، سپس موجودیت‌ها، ویژگی‌ها، کلیدهای اصلی، کلیدهای خارجی، روابط و کاردینالیتی‌ها مشخص شدند. نتیجه این فاز یک طراحی رابطه‌ای برای PostgreSQL بود که قابلیت‌های رزرو، پرداخت، کنسلی، refund، گزارش مشکل، کیف پول، OTP و جزئیات اختصاصی رشته‌های ورزشی را پشتیبانی می‌کند.

۴-۱. اهداف فاز اول

- شناسایی موجودیت‌های اصلی و فرعی سیستم بر اساس نیازمندی‌های پروژه.
- تعیین فیلدهای هر جدول، نوع داده، کلید اصلی و کلید خارجی.
- طراحی روابط یک‌به‌چند، یک‌به‌یک و رابطه‌های اختیاری بین موجودیت‌ها.
- طراحی ساختاری که بعدها بتواند در Backend، SQL و UI بدون تغییر اساسی استفاده شود.
- رعایت اصول نرمال‌سازی برای جلوگیری از تکرار داده و خطاهای به‌روزرسانی.

۴-۲. موجودیت‌های نهایی دیتابیس

در طراحی نهایی، ۲۱ جدول در دیتابیس وجود دارد. بعضی جدول‌ها هسته اصلی سیستم هستند و بعضی دیگر برای عملیات خرید، پشتیبانی، امنیت و امکانات تکمیلی استفاده می‌شوند.

نام جدول	نقش در سیستم	توضیح
----------	--------------	-------

users	کاربران سیستم	تماشاگر و پشتیبان، اطلاعات ورود، شهر، وضعیت، رمز هش شده
cities	شهرها	شهر و استان برای محل سکونت کاربر و مکان ورزشگاه
venues	ورزشگاه‌ها و سالن‌ها	نام، آدرس، ظرفیت و شهر محل برگزاری
sports	رشته‌های ورزشی	فوتبال، والیبال، بسکتبال و رشته‌های قابل توسعه
organizers	برگزارکنندگان	مجموعه یا سازمان برگزارکننده مسابقه
seat_categories	رده‌های جایگاه	VIP، Premium، Family، Normal و توضیحات امکانات
matches	مسابقات	تیم میزبان، تیم مهمان، تاریخ، ساعت، ورزش، ورزشگاه و وضعیت
tickets	بلیط‌ها	قیمت، ظرفیت باقی‌مانده، فعال بودن و ارتباط با مسابقه و جایگاه
football_details	جزئیات بلیط فوتبال	لیگ، شماره بخش/ردیف/صندلی، نوع بلیط و امکانات
volleyball_details	جزئیات بلیط والیبال	سالن، جایگاه، ردیف، صندلی، رده و امکانات
basketball_details	جزئیات بلیط بسکتبال	سالن، جایگاه، ردیف، صندلی، رده و امکانات
cancellation_policies	قوانین کنسلی	مهلت زمانی و درصد جریمه برای محاسبه مبلغ برگشتی
reservations	رزروها	رزرو موقت، زمان انقضا، تعداد، وضعیت پرداخت یا کنسلی
reservation_actions	تاریخچه عملیات رزرو	ثبت رخدادهای مهم رزرو مثل ایجاد، پرداخت، کنسلی
payments	پرداخت‌ها	پرداخت موفق/ناموفق/مسترد، مبلغ، روش و transaction_id
refunds	استرداد وجه	مبلغ برگشتی، وضعیت، دلیل، پشتیبان تاییدکننده

wallets	کیف پول کاربران	موجودی هر کاربر برای پشتیبانی از بازگشت وجه
wallet_transactions	تراکنش کیف پول	ثبت واریز، برداشت، refund و توضیحات مرجع
reports	گزارش مشکلات	گزارش کاربر درباره بلیط یا رزرو و پاسخ پشتیبان
notifications	اعلان‌ها	پیام‌های سیستمی برای کاربر درباره عملیات مهم
otp_logs	لاگ کدهای OTP	ثبت هدف، شناسه کاربر، زمان انقضا و وضعیت کد تایید

۳-۴. توضیح طراحی جدول‌های اصلی

users

جدول کاربران، اطلاعات هویتی و امنیتی کاربر را نگهداری می‌کند. فیلد role مشخص می‌کند کاربر spectator است یا support. برای ورود امن، password_hash ذخیره می‌شود و خود password خام در دیتابیس نگهداری نمی‌شود.

matches

این جدول هسته رویداد ورزشی است و به sport، venue و organizer وصل می‌شود. با این طراحی، اطلاعات ورزش و ورزشگاه تکرار نمی‌شود و هر مسابقه فقط به رکورد مستقل آن‌ها ارجاع می‌دهد.

tickets

بلیط به match و seat_category متصل است و قیمت و ظرفیت باقی‌مانده را نگهداری می‌کند. در زمان رزرو، ظرفیت با transaction کم می‌شود تا race condition ایجاد نشود.

reservations

این جدول رزرو موقت را مدیریت می‌کند. زمان ایجاد رزرو و expiry_time ثبت می‌شود و status نشان می‌دهد رزرو reserved، paid، cancelled یا expired است.

payments

هر پرداخت به یک reservation وصل می‌شود. transaction_id برای شناسایی تراکنش تولید می‌شود و status می‌تواند successful، failed یا refunded باشد.

refunds

در زمان استرداد وجه، با توجه به وضعیت پرداخت و جریمه کنسلی، رکورد refund ساخته می‌شود. این جدول برای گزارش مالی و پیگیری پشتیبانی لازم است.

reports

کاربر می‌تواند درباره بلیط یا رزرو گزارش مشکل ثبت کند. پشتیبان می‌تواند status و پاسخ report را تغییر دهد.

otp_logs

هر OTP علاوه بر Redis در دیتابیس نیز log می‌شود تا مشخص باشد چه کاربری، برای چه هدفی و در چه زمانی کد دریافت کرده است.

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

۴-۴. روابط و کاردینالیتی‌های مهم

رابطه	کاردینالیتی	توضیح
City → Users	1:M	هر شهر می‌تواند چند کاربر داشته باشد، ولی هر کاربر یک شهر دارد.
City → Venues	1:M	هر شهر چند ورزشگاه یا سالن دارد.
Sport → Matches	1:M	هر رشته ورزشی چند مسابقه دارد.
Venue → Matches	1:M	هر ورزشگاه می‌تواند میزبان چند مسابقه باشد.
Match → Tickets	1:M	هر مسابقه چند نوع بلیط با جایگاه و قیمت متفاوت دارد.
SeatCategory → Tickets	1:M	هر رده جایگاه در چند بلیط استفاده می‌شود.
Ticket → Football/Volleyball/Basketball Details	1:0..1	هر بلیط بسته به رشته ورزشی فقط یک نوع جزئیات اختصاصی دارد.
User → Reservations	1:M	هر کاربر می‌تواند چند رزرو داشته باشد.
Reservation → Payment	1:0..1	هر رزرو حداکثر یک پرداخت دارد.
Payment → Refund	1:0..1	هر پرداخت حداکثر یک refund دارد.
User → Reports	1:M	هر کاربر می‌تواند چند گزارش مشکل ثبت کند.
Wallet → WalletTransactions	1:M	هر کیف پول چند تراکنش دارد.

۵-۴. دلیل جداسازی جزئیات رشته‌های ورزشی

در پروژه، بلیط فوتبال، والیبال و بسکتبال ویژگی‌های اختصاصی دارند. اگر همه ویژگی‌ها در جدول tickets قرار می‌گرفت، تعداد زیادی ستون NULL ایجاد می‌شد؛ چون مثلا stadium_name برای فوتبال معنا دارد ولی برای والیبال یا بسکتبال ممکن است hall_name مهم‌تر باشد. به همین دلیل سه جدول volleyball_details، football_details و basketball_details طراحی شد. این کار ساختار tickets را عمومی نگه می‌دارد و توسعه رشته‌های جدید را ساده‌تر می‌کند.

۶-۴. نرمال‌سازی

طراحی دیتابیس تا سطح سوم نرمال یعنی 3NF انجام شده است. هدف از نرمال‌سازی کاهش افزونگی داده، جلوگیری از ناسازگاری و ساده شدن update و delete است.

سطح	رعایت در پروژه	مثال
-----	----------------	------

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

شماره صندلی، ردیف، شهر و ورزش هر کدام در فیلد یا جدول مشخص خود قرار دارند.	تمام فیلدها اتمی هستند و مقدار تکرار شونده در یک ستون ذخیره نشده است.	1NF
اطلاعات شهر در users یا venues تکرار نشده و با city_id ارجاع داده شده است.	وابستگی جزئی به کلید اصلی وجود ندارد و جدول ها هدف مشخص دارند.	2NF
نام ورزشگاه داخل matches ذخیره نشده؛ match فقط venue_id دارد و نام از جدول venues خوانده می شود.	وابستگی انتقالی حذف شده است.	3NF

۷-۴. محدودیت های داده ای و یکپارچگی

- کلیدهای خارجی باعث می شوند رزرو بدون کاربر یا بلیط معتبر ساخته نشود.
- وضعیت ها با enum یا check constraint کنترل می شوند تا مقدار نامعتبر وارد نشود.
- در بلیطها ظرفیت باقی مانده نباید منفی شود و در Backend نیز قبل از رزرو ظرفیت قفل و بررسی می شود.
- برای OTP، وضعیت verified فقط وقتی قابل ثبت است که زمان verified_at نیز مشخص شود.
- در refund، پرداخت باید موفق باشد و برای یک payment دوباره refund تکراری ثبت نشود.

۵. فاز دوم: پیاده سازی SQL، داده اولیه، کوئری ها، توابع و ایندکس ها

پس از طراحی ERD، فاز دوم مربوط به ساخت واقعی دیتابیس در PostgreSQL بود. در این مرحله جداول، enum ها، کلیدها، محدودیت ها، داده اولیه، کوئری های تحلیلی، توابع و ایندکس ها پیاده سازی شدند. نام دیتابیس نهایی پروژه sports_ticketing_db است.

۵-۱. مراحل ساخت دیتابیس

1. ساخت دیتابیس sports_ticketing_db در PostgreSQL.
2. تعریف enum ها و constraint های لازم برای role، status، payment status، reservation status و ticket type.
3. ساخت جدول های پایه مثل cities، sports، organizers و seat_categories.
4. ساخت جدول های وابسته مثل matches، users، venues و tickets.
5. ساخت جدول های عملیاتی شامل wallets، reports، refunds، payments، reservations و wallet_transactions.
6. وارد کردن داده اولیه برای تست کامل سیستم.
7. نوشتن ۲۲ کوئری تحلیلی و ۸ تابع دیتابیسی.
8. اضافه کردن ایندکس ها و بررسی عملکرد query ها با EXPLAIN ANALYZE.

۵-۲. گروهبندی جدول ها در SQL

گروه	جدول ها	هدف
جدول های پایه	cities, sports, organizers, seat_categories	اطلاعات مرجع که چندبار در سیستم استفاده می شود

جدول‌های مکان و رویداد	venues, matches	محل، تاریخ، ساعت و تیم‌های مسابقه
جدول‌های بلیط	tickets, football_details, volleyball_details, basketball_details	قیمت، ظرفیت، جایگاه و جزئیات اختصاصی
جدول‌های کاربر و امنیت	users, otp_logs, notifications	ثبت‌نام، ورود، OTP و پیام‌های سیستمی
جدول‌های رزرو و مالی	reservations, reservation_actions, payments, refunds, wallets, wallet_transactions	رزرو، پرداخت، کنسلی، refund و تاریخچه مالی
جدول‌های پشتیبانی	reports	ثبت گزارش و رسیدگی پشتیبان

۵-۳. توضیح داده اولیه

برای اینکه همه سناریوهای پروژه قابل تست باشند، داده اولیه باید فقط چند رکورد ساده نباشد؛ بلکه باید شهرهای مختلف، ورزشگاه‌های مختلف، رشته‌های ورزشی، مسابقات، جایگاه‌ها، بلیط‌ها، کاربران spectator و support، رزروهای مختلف، پرداخت‌ها، refund و report‌ها را پوشش دهد. به همین دلیل داده اولیه به گونه‌ای طراحی شد که بتوان سناریوهای جستجو، رزرو، پرداخت، کنسلی، refund، تاریخچه و پشتیبانی را تست کرد.

۵-۴. کوئری‌های تحلیلی فاز دوم

کوئری‌های تحلیلی فقط برای نمایش ساده داده نیستند. این کوئری‌ها نشان می‌دهند دیتابیس قابلیت پاسخ به سوالات مدیریتی و عملیاتی سیستم را دارد؛ مثل فروش هر مسابقه، کاربران فعال، رزروهای منقضی‌شده، ظرفیت باقی‌مانده، بلیط‌های کنسل‌شده و گزارش‌های باز.

شماره	موضوع کوئری	کاربرد
۱	نمایش بلیط‌های فعال یک شهر	کمک به فیلتر شهر در صفحه جستجو
۲	نمایش بلیط‌های یک رشته ورزشی	فیلتر بر اساس فوتبال/والیبال/بسکتبال
۳	جستجو بر اساس بازه قیمت	پیدا کردن بلیط‌های اقتصادی یا VIP
۴	نمایش ظرفیت باقی‌مانده هر بلیط	کنترل موجودی قبل از رزرو
۵	گزارش فروش هر مسابقه	تحلیل درآمد و تعداد فروش
۶	پرفروش‌ترین مسابقات	تحلیل محبوبیت رویدادها
۷	کاربران با بیشترین خرید	تحلیل مشتریان فعال
۸	رزروهای منقضی‌شده	پشتیبانی از auto-expire
۹	رزروهای پرداخت‌نشده	کنترل رزروهای موقت
۱۰	پرداخت‌های موفق و ناموفق	تحلیل وضعیت تراکنش‌ها

۱۱	مبالغ refund شده	کنترل مالی استردادها
۱۲	گزارش مشکلات باز	کار پشتیبان برای رسیدگی
۱۳	گزارش فروش به تفکیک شهر	تحلیل فروش مکانی
۱۴	گزارش فروش به تفکیک رشته ورزشی	تحلیل محبوبیت رشته‌ها
۱۵	بلیت‌های VIP و Premium	تحلیل جایگاه‌های خاص
۱۶	تاریخچه رزرو یک کاربر	نمایش history در UI
۱۷	جستجو بر اساس تاریخ مسابقه	فیلتر زمانی
۱۸	ظرفیت باقی‌مانده ورزشگاه‌ها	کنترل موجودی کلی
۱۹	رزروهای cancel شده	تحلیل علت‌های کنسلی
۲۰	پرداخت‌های refunded	ردیابی بازگشت وجه
۲۱	کاربران هم‌شهر	نمونه query تحلیلی روی رابطه User-City
۲۲	گزارش‌های مرتبط با موضوع خاص	تحلیل محتوایی reports

۵-۵. توابع دیتابیس

توابع دیتابیس برای encapsulate کردن منطق‌های پرتکرار استفاده شدند. به جای اینکه هر بار یک query طولانی نوشته شود، با function می‌توان نتیجه مشخص را با پارامتر دریافت کرد. این موضوع هم خوانایی کد را بالا می‌برد و هم برای گزارش پروژه امتیاز دارد.

نام تابع	ورودی اصلی	خروجی/کاربرد
fn_user_purchased_tickets	user_id	نمایش بلیت‌های خریداری‌شده یک کاربر
fn_cancelled_users_by_support	support_id	نمایش کاربرانی که توسط پشتیبان در فرآیند کنسلی بررسی شده‌اند
fn_purchased_tickets_by_city	city_id	گزارش بلیت‌های خریداری‌شده بر اساس شهر
fn_search_tickets	sport/city/date/price	جستجوی پارامتری بلیت‌ها
fn_users_same_city	city_id یا user_id	نمایش کاربران هم‌شهر
fn_top_buyers_since	date	نمایش خریداران برتر از تاریخ مشخص

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

نمایش بلیت‌های کنسل‌شده یک رشته ورزشی	sport_id	fn_cancelled_tickets_by_sport
نمایش کاربران دارای گزارش با موضوع مشخص	subject	fn_users_by_report_subject

۶-۵. ایندکس‌گذاری و بهینه‌سازی

در سیستم رزرو بلیط، بیشترین جستجو‌ها روی فیلدهایی مانند city، sport، match_date، ticket price، capacity_remaining و status انجام می‌شود. برای همین روی فیلدهای پرتکرار و کلیدهای خارجی ایندکس تعریف شد. هدف از ایندکس این است که PostgreSQL هنگام اجرای query به جای full table scan بتواند از index scan یا bitmap index scan استفاده کند.

ناحیه ایندکس	فیلدهای پیشنهادی	دلیل
tickets	match_id, seat_category_id, price, capacity_remaining, is_active	فیلتر و رزرو بلیط
matches	sport_id, venue_id, match_date, status	جستجو و نمایش مسابقات
venues	city_id	فیلتر بر اساس شهر
reservations	user_id, ticket_id, status, expiry_time	history، active reservations و auto-expire
payments	reservation_id, user_id, status, transaction_id	گزارش مالی و refund
reports	user_id, status, subject	پنل پشتیبانی
otp_logs	user_id, identifier, status, expires_at	ثبت و بررسی OTP

```
EXPLAIN ANALYZE
SELECT ...
FROM tickets t
JOIN matches m ON m.match_id = t.match_id
JOIN venues v ON v.venue_id = m.venue_id
WHERE t.is_active = true
      AND v.city_id = 10000000
      AND t.capacity_remaining >= 1;
```

Data Output		
table_name	name	
1	basketball_details	
2	cancellation_polic...	
3	cities	
4	football_details	
5	matches	
6	notifications	
7	organizers	
8	otp_logs	
9	payments	
10	refunds	
11	reports	
12	reservation_actions	
13	reservations	
14	seat_categories	
15	sports	
16	tickets	
17	users	
18	venues	
19	volleyball_details	
20	wallet_transactions	
21	wallets	
Total rows: 21 Query complete 00:00:06.983		

	table_name name	column_name name	data_type character varying	is_nullable character varying (3)
1	basketball_details	basketball_detail_id	integer	NO
2	basketball_details	league_or_tournament	character varying	NO
3	basketball_details	hall_name	character varying	NO
4	basketball_details	section_number	character varying	YES
5	basketball_details	row_number	character varying	YES
6	basketball_details	seat_number	character varying	YES
7	basketball_details	ticket_category	USER-DEFINED	NO
8	basketball_details	facilities	character varying	YES

86	reservation_actions	action_id	integer	NO
87	reservation_actions	reservation_id	integer	NO
88	reservation_actions	action_type	USER-DEFINED	NO
89	reservation_actions	old_status	USER-DEFINED	YES
90	reservation_actions	new_status	USER-DEFINED	NO
91	reservation_actions	action_by_user_id	integer	YES
92	reservation_actions	action_time	timestamp without time zo...	NO
93	reservation_actions	note	character varying	YES
94	reservations	reservation_id	integer	NO
95	reservations	user_id	integer	NO
96	reservations	ticket_id	integer	NO
97	reservations	quantity	integer	NO
98	reservations	reservation_time	timestamp without time zo...	NO
99	reservations	expiry_time	timestamp without time zo...	NO
100	reservations	status	USER-DEFINED	NO
101	reservations	cancelled_at	timestamp without time zo...	YES
102	reservations	cancellation_reason	character varying	YES
103	reservations	updated_at	timestamp without time zo...	NO

	table_name text	row_count bigint
1	cities	10
2	matches	10
3	organizers	10
4	payments	14
5	refunds	12
6	reports	12
7	reservations	16
8	sports	10
9	tickets	30
10	users	12
11	venues	10

۶. فاز سوم: Backend API، احراز هویت، Redis و منطق تراکنشی

فاز سوم پروژه شامل ساخت Backend با Node.js و Express است. طبق نیاز پروژه، برای ارتباط با دیتابیس از ORM استفاده نشده و تمام queryها با کتابخانه pg و SQL خام اجرا شده‌اند. این کار باعث می‌شود دانش SQL و طراحی دیتابیس مستقیم در پروژه دیده شود.

۶-۱. ساختار فولدر Backend

```

backend/
  server.js
  .env
  src/
    config/
      db.js
      redis.js
      elasticsearch.js
    middleware/
      auth.js
      role.js
      errorHandler.js
    routes/
      auth.routes.js
      user.routes.js
      cityVenue.routes.js
      ticket.routes.js
      reservation.routes.js
      payment.routes.js
      refund.routes.js
      report.routes.js
      support.routes.js
      search.routes.js
    controllers/
      auth.controller.js
      user.controller.js

```

ticket.controller.js
 reservation.controller.js
 payment.controller.js
 refund.controller.js
 report.controller.js
 support.controller.js
 search.controller.js
 jobs/
 reservationExpiry.job.js
 docs/
 API_DOCUMENTATION.md

۶-۲. نقش server.js

فایل server.js نقطه شروع Backend است. این فایل dotenv را برای خواندن تنظیمات env فعال می‌کند، Express app را می‌سازد، middlewareهای عمومی مثل express.json، cors، helmet، morgan را اضافه می‌کند، routeها را mount می‌کند و در نهایت Redis را وصل کرده و سرور را روی port 5001 اجرا می‌کند. همچنین job مربوط به منقضی کردن رزروهای پرداخت‌نشده از همین نقطه شروع می‌شود.

۶-۳. اتصال به PostgreSQL

اتصال به دیتابیس با Pool انجام می‌شود. استفاده از Pool باعث می‌شود برای هر request اتصال جدید سنگین ساخته نشود و Backend بتواند چند درخواست را همزمان مدیریت کند. تابع query یک wrapper ساده است که SQL و پارامترها را می‌گیرد و نتیجه را برمی‌گرداند.

```
const { Pool } = require("pg");
const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD
});
```

۶-۴. احراز هویت دو مرحله‌ای OTP → Password

در نسخه نهایی، ورود کاربر دو مرحله دارد. ابتدا کاربر identifier و password را وارد می‌کند. identifier می‌تواند email یا phone باشد. Backend کاربر را پیدا می‌کند، password را با bcrypt.compare بررسی می‌کند و فقط اگر password درست باشد OTP تولید می‌کند. OTP در Redis با زمان انقضا ذخیره می‌شود. سپس کاربر OTP را وارد می‌کند و اگر کد درست و منقضی نشده باشد، JWT صادر می‌شود.

مرحله	Endpoint	توضیح
۱	POST /api/auth/login	password و identifier دریافت می‌شود. اگر password معتبر باشد، OTP ساخته می‌شود.
۲	Redis	OTP با کلید otp:password_login:identifier

TTL مثلا ۳۰۰ ثانیه ذخیره می‌شود.		
در دیتابیس log ثبت می‌شود تا فرآیند OTP قابل پیگیری باشد.	otp_logs	۳
OTP کاربر با Redis مقایسه می‌شود.	POST /api/auth/login/verify-otp	۴
اگر OTP صحیح باشد، token حاوی email، user_id و role ساخته می‌شود.	JWT	۵

Login Flow:

User enters email/password

→ Backend validates password

→ Redis stores OTP for 300 seconds

→ User enters OTP

→ Backend verifies OTP

→ JWT token is returned

مزیت این روش این است که حتی اگر password درست باشد، بدون OTP کاربر وارد سیستم نمی‌شود. از طرف دیگر OTP فقط بعد از تایید password ارسال می‌شود و ارسال کد برای کاربر نامعتبر انجام نمی‌شود.

۵-۶. JWT و Role-based Access

بعد از login موفق، token در Frontend داخل localStorage ذخیره می‌شود. در درخواست‌های بعدی، token در Header ارسال می‌شود. Middleware احراز هویت token را verify می‌کند و اطلاعات کاربر را داخل req.user قرار می‌دهد. برای بخش support، یک middleware جدا نقش کاربر را بررسی می‌کند تا فقط role برابر support اجازه دسترسی داشته باشد.

Authorization: Bearer JWT_TOKEN

۶-۶. Redis در پروژه

Redis دو نقش اصلی دارد. نقش اول ذخیره موقت OTP است؛ چون OTP باید بعد از چند دقیقه خودکار منقضی شود. نقش دوم cache کردن داده‌های پرتکرار مثل cities یا نتیجه ticket search است. استفاده از Redis باعث کاهش فشار روی دیتابیس و افزایش سرعت پاسخ می‌شود.

کلید Redis	کاربرد	زمان انقضا
otp:password_login:<identifier>	ذخیره OTP بعد از password login	OTP_TTL_SECONDS، مثلا ۳۰۰ ثانیه
cities:all	cache لیست شهرها	CACHE_TTL_SECONDS، مثلا ۶۰ ثانیه
tickets:*	cache نتایج جستجوی بلیط	پس از تغییر رزرو یا ظرفیت پاک می‌شود
ticket:<id>	cache جزئیات بلیط	در صورت تغییر ظرفیت پاک می‌شود

۷-۶. جستجوی بلیط با فیلترهای پیشرفته

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

Endpoint جستجوی بلیط به صورت dynamic SQL نوشته شده است. یعنی اگر کاربر فقط city را انتخاب کند، فقط شرط city به query اضافه می‌شود. اگر date، seat category، price range یا sport، minimum capacity هم انتخاب شود، شرط‌های مربوط به آن‌ها نیز اضافه می‌شود. این روش باعث می‌شود یک endpoint بتواند چندین حالت جستجو را پوشش دهد.

GET

```
/api/tickets/search?city_id=100000000&sport_id=200000000&seat_category_id=300000000&min_price=100000&max_price=30000000&match_date=2026-08-09&min_capacity=1
```

۸-۶. ایجاد رزرو موقت

ایجاد رزرو یکی از حساس‌ترین قسمت‌های پروژه است، چون ظرفیت بلیط نباید به اشتباه منفی شود. برای همین در Backend از transaction استفاده شده است. ابتدا ticket با SELECT ... FOR UPDATE قفل می‌شود، سپس ظرفیت بررسی می‌شود. اگر ظرفیت کافی باشد، رکورد reservation ساخته می‌شود و capacity_remaining کم می‌شود. اگر هر خطایی رخ دهد transaction rollback می‌شود.

مرحله	عملیات
۱	دریافت ticket_id و quantity از request
۲	شروع transaction
۳	قفل کردن ticket با FOR UPDATE
۴	بررسی فعال بودن ticket و ظرفیت کافی
۵	ساخت reservation با status reserved و expiry_time
۶	کم کردن capacity_remaining
۷	commit transaction و پاک کردن cache بلیط‌ها

۹-۶. Auto-expire رزروهای پرداخت‌نشده

اگر کاربر رزرو را پرداخت نکند، نباید ظرفیت برای همیشه قفل بماند. برای حل این مسئله، job پس‌زمینه هر چند ثانیه رزروهایی را پیدا می‌کند که status آن‌ها reserved است و expiry_time آن‌ها گذشته است. سپس status را expired می‌کند و ظرفیت را به ticket برمی‌گرداند.

```
UPDATE reservations
SET status = 'expired', updated_at = CURRENT_TIMESTAMP
WHERE status = 'reserved'
AND expiry_time < CURRENT_TIMESTAMP;
```

۱۰-۶. پرداخت

پرداخت فقط برای reservation با status reserved مجاز است. اگر رزرو منقضی شده یا cancel شده باشد، پرداخت انجام نمی‌شود. در پرداخت موفق، رکوردی در payments ایجاد می‌شود، transaction_id ساخته می‌شود و status رزرو به paid تغییر می‌کند.

۱۱-۶. Refund و محاسبه جریمه کنسلی

برای refund ابتدا بررسی می‌شود payment موفق وجود داشته باشد و قبلاً refund برای آن پرداخت ثبت نشده باشد. سپس بر اساس فاصله زمانی تا مسابقه درصد جریمه محاسبه می‌شود. اگر زمان زیادی تا مسابقه مانده باشد جریمه کمتر است و اگر زمان کم باشد درصد جریمه بیشتر می‌شود. در نهایت payment به refunded و reservation به cancelled تغییر می‌کند.

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

زمان تا مسابقه	درصد جریمه در منطق Backend
۴۸ ساعت یا بیشتر	٪۰
بین ۲۴ تا ۴۸ ساعت	٪۲۰
کمتر از ۲۴ ساعت	٪۵۰

۱۲-۶. Change Ticket / Change Seat

قابلیت تغییر جایگاه برای رزروهای پرداخت نشده اضافه شد. کاربر reservation_id و new_ticket_id را وارد می‌کند. Backend ابتدا رزرو فعلی را پیدا و قفل می‌کند، سپس بلیط جدید را بررسی می‌کند. بلیط جدید باید با بلیط فعلی متفاوت باشد، ظرفیت کافی داشته باشد و برای همان match باشد. سپس ظرفیت بلیط قبلی برگردانده و ظرفیت بلیط جدید کم می‌شود و reservation به ticket جدید منتقل می‌شود.

شرط	علت
رزرو باید متعلق به کاربر باشد	جلوگیری از تغییر رزرو دیگران
رزرو باید unpaid / reserved باشد	بعد از پرداخت، تغییر مستقیم جایگاه منطقی نیست و باید از مسیر refund/پشتیبانی انجام شود
بلیط جدید باید با بلیط فعلی متفاوت باشد	تغییر واقعی باید رخ دهد
بلیط جدید باید برای همان match باشد	Change seat یعنی تغییر جایگاه همان مسابقه، نه تغییر رویداد
ظرفیت کافی باید وجود داشته باشد	جلوگیری از overbooking

۱۳-۶. Support Panel Backend

برای پشتیبان دو بخش اصلی پیاده‌سازی شده است. اول مدیریت گزارش‌ها که support می‌تواند report‌های کاربران را ببیند، پاسخ بدهد و status را تغییر دهد. دوم مدیریت reservation که support می‌تواند رزروها را بر اساس status فیلتر کند و در صورت نیاز وضعیت یک reservation را تغییر دهد.

۱۴-۶. مستند endpoint‌های Backend

کاربرد	دسترسی	Endpoint
بررسی روشن بودن سرور	عمومی	GET /api/health
تست اتصال PostgreSQL	عمومی	GET /api/health/db
تست اتصال Redis	عمومی	GET /api/health/redis
ثبت‌نام کاربر spectator و ساخت JWT	عمومی	POST /api/auth/signup

بررسی password و identifier، سپس ساخت OTP	عمومی	POST /api/auth/login
تایید OTP بعد از password و صدور JWT	عمومی	POST /api/auth/login/verify-otp
ارسال OTP مستقل برای login یا تست	عمومی/دمو	POST /api/auth/send-otp
تایید OTP مستقل و صدور JWT	عمومی/دمو	POST /api/auth/verify-otp
دریافت اطلاعات کاربر لاگین شده	JWT	GET /api/auth/me
نمایش پروفایل کاربر	JWT	GET /api/users/profile
ویرایش نام، تلفن و شهر	JWT	PATCH /api/users/profile
لیست شهرها، قابل cache در Redis	عمومی	GET /api/cities
لیست ورزشگاه‌ها با فیلتر شهر	عمومی	GET /api/venues
لیست رشته‌های ورزشی برای dropdown	عمومی	GET /api/sports
لیست دسته‌بندی جایگاه برای فیلتر پیشرفته	عمومی	GET /api/seat-categories
جستجوی دقیق با فیلترهای city/sport/seat/price/date/capacity	عمومی	GET /api/tickets/search
جزئیات کامل یک بلیط	عمومی	GET /api/tickets/:id
ایجاد رزرو موقت و کم کردن ظرفیت	JWT	POST /api/reservations
رزروهای فعال کاربر	JWT	GET /api/reservations/active
تاریخچه رزرو، پرداخت و استرداد	JWT	GET /api/reservations/history
تغییر جایگاه/بلیط برای رزرو پرداخت نشده	JWT	POST /api/reservations/:id/change-ticket
محاسبه جریمه کنسلی	JWT	GET /api/reservations/:id/cancellation-penalty
کنسل کردن رزرو و برگرداندن ظرفیت	JWT	POST /api/reservations/:id/cancel
پرداخت رزرو فعال و تغییر status به paid	JWT	POST /api/payments
ثبت refund برای پرداخت موفق	JWT	POST /api/refunds

ثبت گزارش مشکل توسط کاربر	JWT	POST /api/reports
نمایش گزارش های کاربر	JWT	GET /api/reports/my
لیست گزارش ها برای پشتیبان	JWT + support	GET /api/support/reports
تغییر وضعیت و پاسخ به گزارش	JWT + support	PATCH /api/support/reports/:id
مدیریت و فیلتر رزروها توسط پشتیبان	JWT + support	GET /api/support/reservations
تغییر status رزرو توسط پشتیبان	JWT + support	PATCH /api/support/reservations/:id/status
ایندکس کردن بلیط ها در Elasticsearch	عمومی/مدیریتی	POST /api/search/index-tickets
جستجوی متنی Elasticsearch و autocomplete	عمومی	GET /api/search/tickets?q=

۷. فاز چهارم: رابط کاربری، جستجوی پیشرفته، Autocomplete و Elasticsearch

در فاز چهارم برای پروژه یک رابط کاربری کامل ساخته شد. هدف UI فقط نمایش فرم خام نبود، بلکه یک داشبورد حرفه‌ای با منوی کناری، کارت‌های نتیجه، section‌های جداگانه، وضعیت کاربر لاگین‌شده، خروجی‌های خوانا و فرم‌های تست همه قابلیت‌های Backend طراحی شد.

۱-۷. ساختار Frontend

```
frontend/  
  index.html  
  style.css  
  app.js  
  assets/  
    home-image.png
```

فایل index.html ساختار صفحات و فرم‌ها را نگهداری می‌کند. فایل style.css ظاهر حرفه‌ای، رنگ‌ها، کارت‌ها، sidebar، دکمه‌ها و حالت responsive را مدیریت می‌کند. فایل app.js منطق ارتباط با API، ذخیره token، رندر نتایج و مدیریت section‌ها را انجام می‌دهد.

۲-۷. بخش‌های اصلی UI

بخش	قابلیت‌ها
Home	نمایش نام پروژه، وضعیت کاربر، تصویر Home و معرفی سیستم
Authentication	Signup، Password + OTP Login و ذخیره token
Profile	نمایش و ویرایش اطلاعات کاربر
Tickets	جستجوی پیشرفته، نمایش کارت‌های بلیط و جزئیات بلیط
Booking	رزرو، تغییر جایگاه، پرداخت و refund
History	نمایش تاریخچه رزرو، پرداخت و refund کاربر
Support	مدیریت report‌ها و reservation‌ها برای کاربر support
ElasticSearch	Index Tickets، جستجوی متنی و autocomplete

۳-۷. جستجوی دقیق PostgreSQL در UI

در بخش Ticket Search، کاربر می‌تواند فیلترهای دقیق انتخاب کند. city، sport و seat category به صورت dropdown از API خوانده می‌شوند تا کاربر مجبور نباشد id‌ها را حفظ کند. فیلترهای min capacity، min price، max price، match date نیز input هستند. وقتی کاربر search می‌زند، app.js یک query string می‌سازد و آن را به api/tickets/search/ ارسال می‌کند.

فیلتر UI	پارامتر API	اثر روی SQL
City	city_id	شرط c.city_id

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

s.sport_id شرط	sport_id	Sport
sc.seat_category_id شرط	seat_category_id	Seat Category
t.price <= مقدار شرط	min_price	Min Price
t.price >= مقدار شرط	max_price	Max Price
m.match_date شرط	match_date	Match Date
t.capacity_remaining <= مقدار شرط	min_capacity	Min Capacity

۴-۷. جستجوی متنی Elasticsearch

در کنار جستجوی دقیق PostgreSQL، یک جستجوی متنی با Elasticsearch نیز پیاده‌سازی شد. ابتدا با endpoint `api/search/index-tickets/` بلیط‌های فعال از دیتابیس خوانده و داخل index به نام `tickets` ذخیره می‌شوند. سپس کاربر می‌تواند با عبارت متنی مثل `Tehran`، `VIP`، `Azadi`، `Esteghlal` یا `Football` جستجو کند. Elasticsearch روی چند field مختلف جستجو می‌کند و نتیجه‌ها را بر اساس score مرتب می‌کند.

فیلد در Elasticsearch	دلیل جستجو
home_team / away_team	جستجو با نام تیم‌ها
sport_name	جستجو با رشته ورزشی
venue_name	جستجو با نام ورزشگاه یا سالن
city_name / province	جستجو با شهر و استان
league_or_tournament	جستجو با لیگ یا تورنمنت
facilities	جستجو با امکانات جایگاه
seat_category	جستجو با Family، Premium، VIP و غیره

۵-۷. Autocomplete

Autocomplete در UI باعث می‌شود وقتی کاربر در بخش Elasticsearch شروع به تایپ می‌کند، پیشنهادهای مرتبط نمایش داده شود. این قابلیت تجربه کاربری را بهتر می‌کند و نشان می‌دهد Elasticsearch فقط برای search معمولی استفاده نشده، بلکه برای جستجوی هوشمندتر در رابط کاربری نیز به کار رفته است.

۶-۷. نمایش خروجی‌ها به شکل کارت

در نسخه اولیه خروجی‌ها شبیه JSON خام بودند، اما در نسخه نهایی نتیجه‌ها به صورت کارت‌های خوانا نمایش داده می‌شوند. برای بلیط‌ها اطلاعات مهم مثل `ticket_id`، تیم‌ها، شهر، ورزشگاه، قیمت، ظرفیت و `seat category` نمایش داده می‌شود. برای خطاها نیز کارت `error` و برای موفقیت‌ها کارت `success` استفاده می‌شود.

۷-۷. Elasticsearch و Docker

Elasticsearch با Docker اجرا می‌شود تا نیازی به نصب مستقیم روی سیستم نباشد. در docker-compose.yml سرویس elasticsearch با نسخه 8.15.3، single-node و xpack security disabled تعریف شده است. با اجرای docker compose up -d سرویس روی پورت 9200 فعال می‌شود.

```
docker compose up -d
curl http://localhost:9200
```

۸. تست نهایی پروژه و سناریوهای اجرا

برای اطمینان از صحت پروژه، سناریوهای مختلف در Redis، PostgreSQL، UI، Backend، Elasticsearch تست شدند. جدول‌های زیر نشان می‌دهند هر بخش چگونه تست شده و نتیجه مورد انتظار چیست.

۸-۱. تست‌های Backend و API

سناریو	ورودی نمونه	نتیجه مورد انتظار
Health check	GET /api/health	success true و روشن بودن سرور
PostgreSQL connection	GET /api/health/db	نمایش db و user و وضعیت OK
Redis connection	GET /api/health/redis	اتصال موفق به Redis
Password step	POST /api/auth/login با Test12345	demo_otp و Password verified
OTP step	POST /api/auth/login/verify-otp	user data و JWT token
Ticket search	city/sport/price/date/capacity filters	لیست بلیط‌های match شده
Create reservation	quantity و ticket_id	ساخت reservation و کاهش capacity
Change seat	new_ticket_id و reservation_id	تغییر ticket_id رزرو و اصلاح ظرفیت‌ها
Payment	method و reservation_id	پرداخت موفق و status paid
Refund	reason و payment_id	status refund completed و refunded/cancelled
History	GET /api/reservations/history	نمایش رزروها، پرداخت‌ها و refundها
Report	POST /api/reports	ثبت گزارش مشکل
Support reports	support token	دسترسی مجاز و مدیریت reports
Support reservations	support token	مدیریت reservation status
Elasticsearch index	POST /api/search/index-	indexed_count

	tickets	
نتایج دارای score	GET /api/search/tickets?q=Azadi	Elasticsearch search

۲-۸. تست‌های UI

بخش UI	تست انجام‌شده	نتیجه
Authentication	ورود با email و password، سپس OTP	ورود موفق و نمایش کاربر در sidebar
Tickets	جستجو با city، sport، seat، price، capacity و date	کارت‌های بلیط درست نمایش داده شد
Ticket Detail	وارد کردن ticket_id	جزئیات کامل بلیط نمایش داده شد
Booking	رزرو بلیط	reservation_id ساخته شد
Change Ticket / Seat	تغییر reservation 15000016 به ticket دیگر	success و تغییر ظرفیت
Payment	پرداخت reservation فعال	payment_id ساخته شد
Refund	refund یک payment موفق	refund completed
History	نمایش تاریخچه کاربر	رزروها و پرداخت‌ها نمایش داده شد
Support	login با support1@example.com	دسترسی به reports و reservations
Elasticsearch	Index و search	نتیجه search و پیشنهاد autocomplete

۳-۸. خطاهای مهمی که در طول پروژه رفع شدند

مشکل	علت	رامحل
Redis connection refused	Redis اجرا نشده بود	اجرای redis-server یا service
Port already in use	سرور قبلی روی 5001/5000 روشن بود	تغییر PORT یا بستن process قبلی
OTP verify constraint error	verified_at هنگام status verified ست نمی‌شد	اضافه کردن verified_at=CURRENT_TIMESTAMP
Elasticsearch media-type error	عدم سازگاری نسخه client با Elasticsearch 8	استفاده از elastic/elasticsearch @ v8.19.2

بازنویسی dynamic conditions در ticket.controller	شرط city/sport در SQL کامل نبود	Ticket filters درست عمل نمی‌کرد
اصلاح Body و robust parsing در controller	Body JSON یا status درست ارسال نشده بود	Support update status body invalid
ساخت hash با bcrypt و update users	password_hash برای userهای قدیمی ست نبود	Password login invalid credentials
استفاده از ticket دیگر برای همان match	new_ticket_id برابر ticket فعلی بود	Change ticket same ticket error

۹. راهنمای نصب و اجرای پروژه

۹-۱. اجرای Backend

```
cd /Users/arya/Desktop/pr_db4042/backend
npm install
npm run dev
```

در صورت اجرای صحیح، پیام‌های اتصال Redis و اجرای سرور روی port 5001 نمایش داده می‌شود.

۹-۲. فایل env.

```
PORT=5001
NODE_ENV=development
DB_HOST=localhost
DB_PORT=5432
DB_NAME=sports_ticketing_db
DB_USER=postgres
DB_PASSWORD=your_postgres_password
JWT_SECRET=arya_db_project_secret_4042
JWT_EXPIRES_IN=1d
REDIS_URL=redis://127.0.0.1:6379
OTP_TTL_SECONDS=300
CACHE_TTL_SECONDS=60
RESERVATION_TTL_MINUTES=10
RESERVATION_EXPIRY_CHECK_SECONDS=60
ELASTICSEARCH_URL=http://localhost:9200
```

۹-۳. اجرای Redis

```
redis-server
# or
brew services start redis
```

۹-۴. اجرای Elasticsearch

```
cd /Users/arya/Desktop/pr_db4042
docker compose up -d
curl http://localhost:9200
```

۹-۵. اجرای Frontend

گزارش کامل پروژه پایگاه داده - سامانه رزرو بلیط ورزشی

Frontend با Live Server در VS Code اجرا می‌شود. مسیر فایل index.html داخل فولدر frontend است. پس از تغییر فایل‌ها، برای اطمینان از حذف cache باید صفحه با `Cmd + Shift + R` رفرش شود.

۱۰. جمع‌بندی و کارهای قابل توسعه

در این پروژه یک سیستم کامل رزرو و خرید بلیط ورزشی از مرحله طراحی دیتابیس تا UI پیاده‌سازی شد. در فاز اول، موجودیت‌ها، روابط و ERD طراحی شدند. در فاز دوم، دیتابیس PostgreSQL با جداول، داده اولیه، کوئری‌ها، توابع و ایندکس‌ها ساخته شد. در فاز سوم، Backend بدون ORM با احراز هویت JWT، ورود دو مرحله‌ای Password و Redis، OTP، رزرو موقت، پرداخت، refund، support panel و background job ساخته شد. در فاز چهارم، UI حرفه‌ای با جستجوی پیشرفته، تغییر جایگاه، تاریخچه، support panel، Elasticsearch و autocomplete پیاده‌سازی شد.

۱-۱۰. نقاط قوت پروژه

- پوشش کامل چرخه رزرو تا پرداخت و refund.
- استفاده از transaction و SELECT FOR UPDATE برای جلوگیری از خطای ظرفیت.
- ورود امن‌تر با Password + OTP و JWT.
- استفاده از Redis برای OTP و cache.
- استفاده از Elasticsearch برای full-text search و autocomplete.
- UI کامل با نقش‌های spectator و support.
- مستندسازی endpointها و آماده بودن پروژه برای ارائه.

۲-۱۰. توسعه‌های آینده

- اضافه کردن upload عکس پروفایل کاربر.
- پیاده‌سازی تغییر رمز عبور داخل UI.
- اتصال واقعی به درگاه پرداخت به جای payment demo.
- ارسال واقعی OTP با SMS یا Email.
- اضافه کردن پنل ادمین جدا از support.
- Dockerize کامل Backend، Redis و PostgreSQL.
- نوشتن تست خودکار با Jest یا Supertest.